

Day 1 :

- Set zero Matrix
- Pascal triangle
- Next permutation

1. Set zero matrix

Problem Statement: Given a matrix if an element in the matrix is 0 then you will have to set its entire column and row to 0 and then return the matrix.

Approach :

In brute force we use two extra vector, row and col

1. First by nested loop find zero valued element we mark row and column 1 or true ;
2. Next by another nested loop we will track those elements which have marked true or 1 in row and col and set them to 0;

Optimal

Main Idea

Use the first row and first column as markers instead of creating extra arrays.

Steps

1. Traverse the matrix.
2. Whenever a 0 is found:
 - a. Mark its row using first column.
 - b. Mark its column using first row.
3. Traverse again and set cells to 0 using those markers.
4. Handle first row and first column separately.

Pseudocode

For every cell

 If cell is 0

 Mark row

 Mark column

Use markers to fill zeros

Handle first row and first column separately

Important Edge Case

matrix[0][0] cannot represent both first row and first column.

Therefore use:

```
bool firstCol = true;
```

or

```
int col0 = 1;
```

to handle the first column separately.

Why does Optimal Work?

Instead of using extra row[] and col[] arrays,
we use the first row and first column
to store marking information.

Complexity

Time: $O(N \times M)$

Space: $O(1)$

Key Interview Point

Use matrix itself as storage.

2. Pascal Triangle

Approach

Every element is made from:

Upper Left + Upper Right

Example:

```
1
1 1
1 2 1
1 3 3 1
```

Because:

$$2 = 1 + 1$$

$$3 = 1 + 2$$

$$3 = 2 + 1$$

Finding Nth Row

Steps

1. First element is always 1.
2. Generate the next element using the previous element.

Pseudocode

ans = 1

Print ans

For col = 1 to row

 ans = ans * (row - col + 1)

 ans = ans / col

Print ans

Complexity

Time: $O(N)$

Space: $O(1)$

3. Next Permutation

Approach

Goal: Find the next bigger arrangement of the array.

Steps

1. Start from the right side.
2. Find the first element which is smaller than its next element.
 - a. This is called the **pivot**.
3. Again from the right side, find the first element greater than the pivot.
4. Swap them.
5. Reverse the remaining right part.

Example

1 2 5 4 3

Pivot = 2

Swap with 3:

1 3 5 4 2

Reverse right part:

1 3 2 4 5

Pseudocode

Find pivot from right

If pivot exists

Find next greater element

Swap them

Reverse remaining part

Important Edge Case

If no pivot is found,

$\text{arr}[i] < \text{arr}[i+1]$

never occurs.

Example:

5 4 3 2 1

Then reverse the whole array.

Output:

1 2 3 4 5

Add this "Why It Works?"

We find the first position from the right where we can make the number slightly larger.

Then we make the remaining suffix as small as possible by reversing it.

This gives the immediate next permutation.

Complexity

Time: $O(N)$

Space: $O(1)$